

GLS Software Stack — Technical Architecture

Genomic Ledger Synthesis — Sage Clokey — June 2026

The software required to make Genomic Ledger Synthesis work. Most of it extends what already exists in the Adaptive Genome Design System.

1. Compatibility Engine (partially built)

The gatekeeper — checks whether a candidate block can integrate with the current chassis before any wet lab work begins.

- **Codon usage distance calculator** — computes Euclidean distance between 61-dimensional RSCU vectors of the candidate block and the chassis organism. Already implemented in the Adaptive Genome Design System.
- **Regulatory grammar scanner** — checks promoter, RBS, and terminator compatibility between the block and the host regulatory environment. Flags mismatches in transcription factor binding sites, ribosome binding strength, and terminator efficiency.
- **Epistasis predictor** — screens for known antagonistic gene-gene interactions between the candidate block's gene products and existing genome components. Uses published synthetic lethality databases (Keio collection, SGA data) and metabolic network topology.
- **Trade cost calculator** — weighted composite score: $C_{trade} = 0.5 * d_{codon} + 0.3 * R_{regulatory} + 0.2 * B_{baseline}$. Blocks below threshold proceed to integration. Blocks above threshold undergo codon harmonization (not full optimization — preserves local knowledge in rare codons).

Dependencies: Python, NumPy, existing RSCU code from Adaptive Genome Design System

2. Shadow Price Calculator

The economist — tells you what the chassis genome's metabolic economy needs most, so you add the right block next.

- **FBA wrapper** — loads the chassis organism's genome-scale metabolic model (e.g., iML1515 for E. coli, iMM904 for yeast) via COBRApy and runs Flux Balance Analysis.

- **Shadow price extraction** — pulls dual variables from the LP solution. Each shadow price reports the marginal growth value of one metabolite constraint — how much biomass production would improve if that metabolite were more available.
- **Block ranking** — matches each candidate block's output profile (what metabolites it produces) against the shadow price vector. Blocks that produce high-shadow-price metabolites are ranked first. This is Hayekian price-guided resource allocation applied to genome construction.
- **Dynamic re-computation** — after every accepted block integration, FBA re-runs on the updated model. Shadow prices change as the economy changes. The priority queue refreshes automatically.

Dependencies: COBRApy, genome-scale models (BiGG database), SciPy

3. Block Library Manager

The warehouse — a structured database of validated genetic modules ready for integration.

Each block is stored with:

- **Functional metadata** — inputs (substrates consumed, signals read), outputs (products generated, signals emitted), ATP cost, pathway membership
- **Sequence properties** — RSCU vector, GC content, length, codon adaptation index
- **Regulatory elements** — promoter type, RBS strength, terminator, flanking sequences
- **Validation status** — cell-free expression confirmed (yes/no), in vivo validation (which chassis, growth data)
- **Provenance** — source organism, original gene IDs, modification history
- **Hash** — k-mer frequency fingerprint (e.g., 4-mer spectrum) for integrity checking and duplicate detection
- **Integration hardware** — pre-designed homology arms for target locus, LoxPsym flanking sites, unique barcode watermark

Dependencies: SQLite or PostgreSQL, Python, BioPython for sequence handling

4. Ledger Tracker

The blockchain — an append-only record of every integration attempt, successful or failed.

Each ledger entry records:

- **Block ID** — which module was attempted
- **Node ID** — which independent cell population performed the validation
- **Genome state** — hash of the genome before integration (chain link to previous state)
- **Validation metrics** — growth rate, doubling time, metabolite profiling results, transcriptomic confirmation
- **Outcome** — accepted / rejected / pending
- **Timestamp** — when the integration was attempted
- **Junction sequences** — exact genomic coordinates where the block was inserted
- **SCRaMbLE events** — any post-integration rearrangements detected (deletions, inversions, translocations)

The ledger is append-only. Failed integrations are not deleted — they inform future compatibility predictions. The full assembly history is preserved, creating a traceable provenance chain from genesis block (minimal chassis) to final genome.

Dependencies: SQLite, Python, hashlib for genome state hashing

5. Consensus Engine

The validator — collects results from multiple independent nodes and determines whether a block is accepted into the genome.

- **Node management** — tracks N independent cell populations (default N=5), each maintaining its own copy of the growing genome
- **Validation collection** — receives growth rate, metabolite profile, and stress response data from each node after integration attempt
- **Acceptance rule** — configurable threshold (default: pass in $\geq 3/5$ nodes). A block is accepted when it demonstrates maintained or improved fitness in a majority of independent contexts.
- **Context-dependent flagging** — blocks that pass in some nodes but fail in others are flagged for investigation. These reveal context-dependent compatibility — the block works in some genomic backgrounds but not others, indicating epistatic interactions.
- **Propagation trigger** — accepted blocks are queued for propagation to all nodes (via conjugation, transformation, or genome extraction + retransformation)

Dependencies: Python, pandas for data aggregation, statistical tests (SciPy) for significance

6. Assembly Planner

The architect — takes the top-priority block and generates everything needed to physically build and integrate it.

- **Integration construct design** — generates the full DNA sequence for synthesis: homology arms (50-80 bp flanking the target locus) + LoxPsym sites + block sequence + unique barcode watermark
- **Synthesis order generation** — formats the construct for ordering from Twist Bioscience or IDT (FASTA format, codon-harmonized, synthesis constraints checked)
- **SCRaMbLE scheduler** — after every 5-10 blocks are integrated, schedules a SCRaMbLE round (Cre-LoxPsym activation) followed by selection and sequencing
- **Adaptive evolution scheduler** — after full assembly, generates the protocol for adaptive laboratory evolution (passage conditions, selection pressures, generation targets, sampling schedule)
- **Experiment tracker** — links each physical experiment back to the ledger entry, closing the loop between computational prediction and wet lab result

Dependencies: Python, BioPython, Twist/IDT API specifications

What Can Be Built Right Now From SLO

Component	Dependencies	Status	Effort
Compatibility Engine	Python, existing RSCU code	Extend existing code	2-3 weeks
Shadow Price Calculator	COBRAPy, iML1515	Start from scratch	1-2 weeks
Block Library Manager	SQLite, BioPython	Start from scratch	2-3 weeks
Ledger Tracker	SQLite, hashlib	Start from scratch	1-2 weeks
Consensus Engine	Python, pandas, SciPy	Start from scratch	1-2 weeks
Assembly Planner	BioPython, API specs	Start from scratch	2-3 weeks

Total estimated build time: 10-16 weeks for a working prototype of all six components.

All six are Python. All run on a laptop. None require wet lab access. This is the SBIR deliverable and the MS thesis in one package.

Integration Architecture

The six components connect as a pipeline:

1. **Shadow Price Calculator** scans the chassis economy and ranks candidate blocks
2. **Compatibility Engine** filters the top candidates for regulatory and codon compatibility
3. **Block Library Manager** retrieves the winning block's full specification
4. **Assembly Planner** generates the integration construct and synthesis order
5. **Consensus Engine** collects validation results from multiple nodes after wet lab integration
6. **Ledger Tracker** records the outcome and updates the genome state

Then the cycle repeats: shadow prices are recomputed on the updated genome, the next block is selected, and the ledger grows by one entry. Each cycle adds one validated block to the genome — one transaction confirmed on the distributed ledger.

Technology Stack Summary

Layer	Technology
Language	Python 3.12
Metabolic modeling	COBRApy, BiGG Models
Sequence analysis	BioPython, NumPy
Database	SQLite (prototype) / PostgreSQL (production)
Statistics	SciPy, pandas
Integrity	hashlib (SHA-256 genome state hashing)
Visualization	Matplotlib, Plotly
DNA synthesis interface	Twist Bioscience API, IDT API
Version control	Git + GitHub